

CH 1.2: DDBMS ARCHITECTURES

Contents

Reference Architecture	1
Client-Server Architecture	2
(i) Using DDBMS Primitives.....	2
(ii) Using Auxiliary Program and RPC (Remote Procedure Call)	3
Collaborating Server Architecture	4
Middleware Architecture	4
Peer-to-Peer (P2P) Architecture	5
Multi-DBMS/ Federated Database System Architecture.....	5
Transparency in Distributed Databases.....	7
Data Replication in Distributed Databases.....	10

Reference Architecture

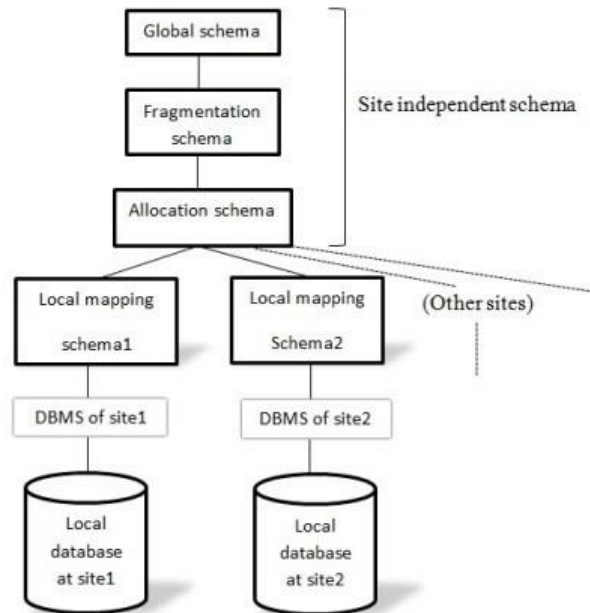
The reference architecture of a Distributed DBMS (DDBMS) defines the logical structure and functional components that enable distributed database management. It provides a standardized framework for designing and implementing DDBMSs. **Components of Reference Architecture:**

- **Global Schema (Conceptual Level):** Represents the overall logical view of the distributed database as a single entity other than showing the database is distributed at all.
Eg: Emp (EID, ENAME, DEPT)
- **Fragmentation Schema:** Defines how data in global schema is divided into fragments (horizontal, vertical, or mixed) and their placement in the distributed system.
Eg: Fragmenting the Employee global schema into:
 - **E1 based in Dept= 'Manager' Emp**
 - **E1 based in Dept= 'Sales Emp**
- **Allocation Schema:** Specifies where each fragment is stored (sites) across different nodes.
Eg: Allocation Schema: E1 at site 2,5 and E2 at site 1,3
- **Local Schemas (Internal Level):** Represents the physical storage and organization of data in each local database. Helps to identify the global relation schema for any local database relation schema.
- **External Schema (User Level):** Defines user views, specifying which part of the database a user or application can access.

Advantages:

- Provides a **structured approach** to designing DDBMS.
- Helps in **efficient query optimization** and transaction management.

- Supports **scalability** and **fault tolerance** by defining clear data distribution rules.



Client-Server Architecture

This architecture involves one or more **clients** that requests data and one or more **servers** that processes and responds to these requests. It is one of the most simple and common models used in DDBMS due to centralized server system. There are two variations:

(i) Using DDBMS Primitives

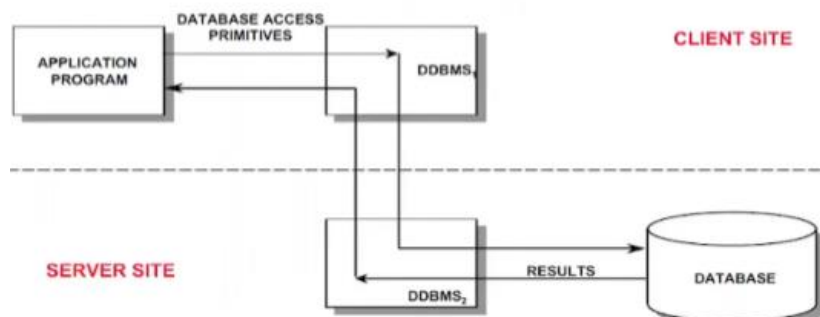
In this approach, clients interact directly with a Distributed DBMS that provides built-in support for distributed database operations. **How it works:**

- There are 2 sites: client site and server site
- The local server of DDBMS at client site routes the query to the server site.
- Clients send **structured queries** (SQL) to the DDBMS.
- The **DDBMS at server site handles query processing**, fragmentation, and data retrieval.
- It accesses the database, resolves the query and sends it back to the client site.

Features:

- Uses **standardized query languages (SQL)** with extensions for distribution.
- Query optimization and transaction management are **handled by the DDBMS itself**.
- High distribution transparency (because file names are global).
- Low efficiency since the answer travels 1 tuple at a time.

USING DDBMS PRIMITIVES



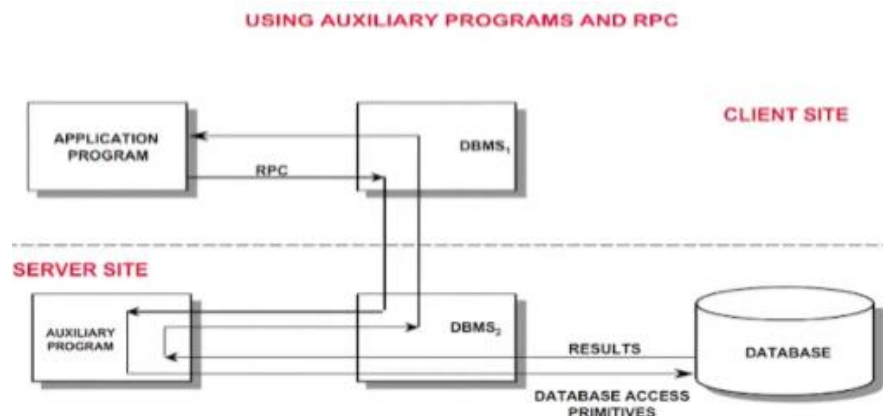
Advantages:

- Easier to manage as the DDBMS **automates** data distribution tasks.
- Provides **better performance** in distributed query execution.
- More **secure and reliable** since transaction control is centralized in the DDBMS.

(ii) Using Auxiliary Program and RPC (Remote Procedure Call)

This approach involves additional auxiliary programs to manage distributed queries and transactions instead of relying solely on the DDBMS. **How it works:**

- The application program at Clients site send requests to an **auxiliary program** that acts as an intermediary.
- The auxiliary program **interprets queries and manages communication** between clients and multiple servers.
- Remote Procedure Calls (RPC) enable **clients to execute procedures on remote servers** as if they were local.
- The auxiliary program assembles tuples into result sets improving transmission efficiency



Advantages:

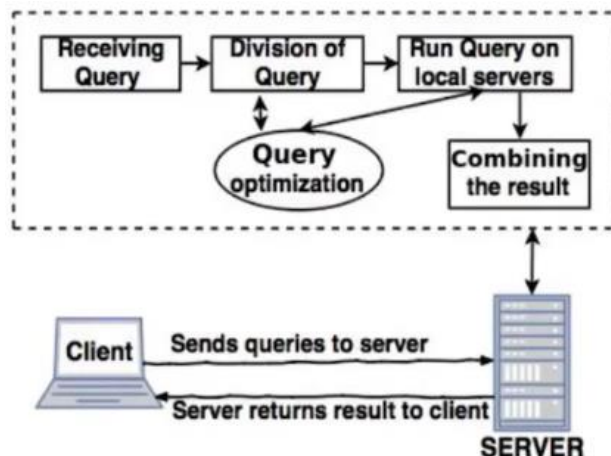
- Reduces the complexity of the DDBMS by delegating tasks to external programs.
- Provides **flexibility** as auxiliary programs can be customized for specific applications.
- RPC-based communication is **efficient** for distributed processing.

Feature	Reference Architecture	Client-Server (DDBMS Primitives)	Client-Server (Auxiliary Program & RPC)
Focus	Standardized structure for distributed DBMS	Direct interaction with DDBMS	Uses intermediary for distributed processing
Query Processing	Defined layers for query optimization	Handled by DDBMS	Managed by auxiliary program
Transaction Management	Defined structure for ACID properties	Centralized in DDBMS	Requires custom handling
Communication	Standardized architecture	Direct SQL queries	RPC-based calls
Flexibility	High (supports multiple models)	Medium (DDBMS-dependent)	Very High (customizable middleware)
Ease of Management	Complex but structured	Easier due to DDBMS automation	More effort needed for custom solutions

Collaborating Server Architecture

In this architecture, multiple database servers work together to process a single query or transaction efficiently. Instead of a single server managing everything, multiple servers **collaborate** to handle distributed data and transactions. **How it Works:**

- Each server has its own **local database** but can also access data from other servers.
- Upon receiving query server breaks the single query into multiple small queries and result is sent to the client.
- A **distributed query processor** manages queries across multiple servers.



Key Features:

- Supports **data distribution, consistency and concurrency across multiple sites**.
- Servers coordinate with each other **without a central controller**.
- Allows **high availability and fault tolerance**.

Advantages:

- **Scalability:** As more servers are added, performance improves.
- **Reliability:** If one server fails, others can still function.
- **Efficient Query Execution:** Queries can be executed in parallel across multiple servers.

Disadvantages:

- **Complex coordination** is required between servers.
- **Increased communication overhead** can impact performance.

Middleware Architecture

This architecture introduces an **intermediate layer (middleware)** between the client and the database servers. **How it Works:**

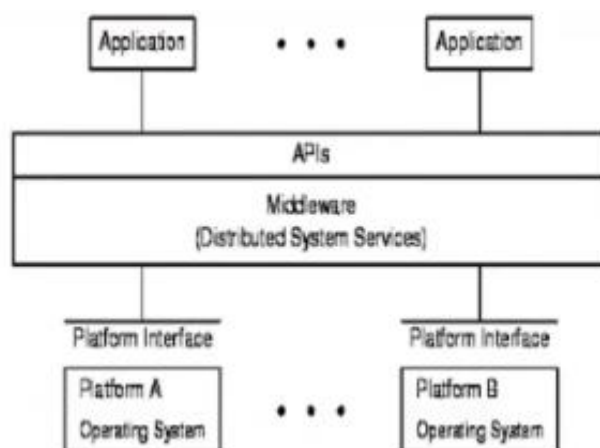
- Single query is executed in multiple servers.
- Clients send queries to the **middleware layer**.
- The middleware processes and optimizes the queries, then forwards them to the appropriate database servers.
- Middleware integrates data from multiple sources and presents a **unified view** to clients.

Key Features:

- Provides **location transparency**: Clients don't need to know where the data is stored.
- Supports **heterogeneous databases** (databases from different vendors).
- Offers **load balancing and query optimization**.

Advantages:

- **Improved interoperability:** Can integrate different types of databases.
- **Better performance:** Middleware optimizes query execution.
- **Security:** Middleware can enforce access controls and authentication.



Disadvantages:

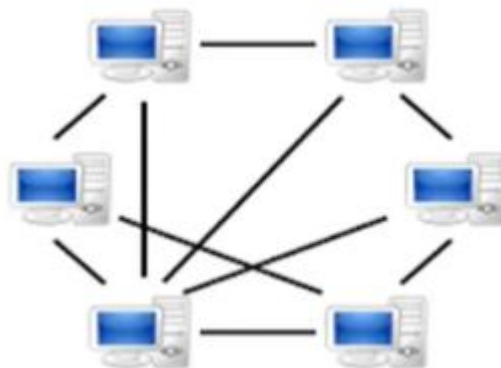
- **Additional processing delays** due to middleware overhead.
- **Complex implementation:** Needs careful configuration to ensure efficiency.

Peer-to-Peer (P2P) Architecture

In a **peer-to-peer** (P2P) architecture, all database nodes act as **equal peers**. There is no centralized control, and each node can function as both a client and a server.

How it Works:

- Each peer stores part of the database and can **process queries independently**.
- Peers communicate directly with each other to **share data and transaction information**.
- No **master-slave** relationship—each peer has equal responsibility.



Key Features:

- **Decentralized:** No central authority or control.
- **Self-organizing:** Peers can join or leave dynamically.
- **Replication and redundancy:** Data is often duplicated across peers to enhance availability.

Advantages:

- **Fault tolerance:** If one peer fails, others continue functioning.
- **Scalability:** New peers can be added easily.
- **Efficient resource utilization:** Workload is distributed across peers.

Disadvantages:

- **Data consistency issues:** Keeping data synchronized across peers is challenging.
- **Security concerns:** No central control makes access management difficult.
- **Higher communication overhead:** Each peer must communicate frequently with others.

Multi-DBMS/ Federated Database System Architecture

Multi-DBMS is designed to integrate multiple databases that may be **heterogeneous (from different vendors)** or **homogeneous (same vendor)**.

How it Works:

- A **federated database management system (FDBMS)** coordinates queries and transactions across multiple databases.
- Each database retains **local autonomy**—they continue operating independently.
- The system can be **loosely coupled (with minimal integration)** or **tightly coupled (with more central control over query execution)**.

The architecture has two major layers:

- A. Global (Multi-Database) Layer**
- B. Local (Individual DBMS) Layer**

1. Multi-Database View Level/Global External Schema.

- Represents multiple **user/application views** over multiple databases.
- Provides a **customized view** hiding system and distribution complexity.

2. Multi-Database Conceptual Level/Global Conceptual Schema (GCS).

- Describes the **logical structure** of the **entire federation**.
- Provides a **global schema** integrating the schemas of all local databases.
- Handles issues like **schema mapping, conflicts, and data integration**.
- Acts as a mediator between global views and local databases.

3. Multi-Database Internal Level/Global Internal Schema.

- Deals with the **physical representation** of the federated database.
- Manages the **distribution details, data fragmentation, replication, and communication** between databases.

4. Local Database View Level/ Local External Schema.

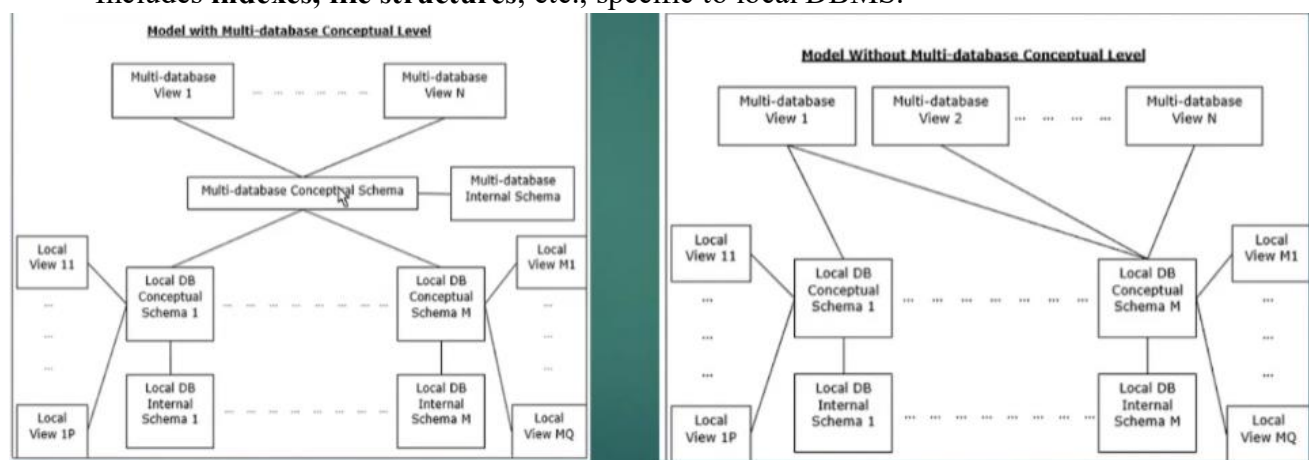
- Represents the **local user views** defined on each autonomous DBMS.
- Specific to individual databases — not aware of the global system.

5. Local Database Conceptual Level/Local Conceptual Schema.

- Defines the **logical structure** of data in each local database.
- Maintained by each local DBMS independently.

6. Local Database Internal Level/ Local Internal Schema.

- Describes **how data is stored** physically at each local site.
- Includes **indexes, file structures, etc.**, specific to local DBMS.



Advantages:

- **Flexibility:** Can integrate databases from different vendors.
- **No need for data migration:** Each database remains in its original format.
- **Better resource utilization:** Databases continue to function independently.

Disadvantages:

- **Query processing is complex:** Requires query transformation across different database types.
- **Performance overhead:** Due to data integration and coordination between databases.
- **Security challenges:** Managing access control across multiple databases is difficult.

Multi-database View Level - Depicts multiple user views comprising of subsets of the integrated distributed database.

Multi-database Conceptual Level - Depicts integrated multi-database that comprises of global logical multi-database structure definitions.

Multi-database Internal Level - Depicts the data distribution across different sites and multi-database to local data mapping.

Local database View Level - Depicts public view of local data.

Local database Conceptual Level - Depicts local data organization at each site.

Local database Internal Level - Depicts physical data organization at each site.

Architecture	Control Type	Scalability	Fault Tolerance	Heterogeneous DB Support	Complexity
Collaborating Server	Distributed	High	High	No	High
Middleware	Centralized Middleware	Medium	Medium	Yes	High
Peer-to-Peer (P2P)	Decentralized	Very High	Very High	No	Very High
Multi-DBMS	Semi-Decentralized	Medium	Medium	Yes	High

Transparency in Distributed Databases

- In distributed systems, the users are unaware of where the data are physically stored and how the data can be accessed at specific local site.
- User should be able to access the database exactly as if the system were local.
- Hiding all details from the users is called data transparency.
- Transparency allows a physically dispersed database to be managed as if it were a centralized database.
- Distribution transparency is the property of distributed databases by the virtue of which internal details of the distribution are hidden from the user.

Types of Transparency in DDBMS

1. **Location Transparency:** Users can access data without knowing where it is physically stored. The database system automatically determines the best site from which to retrieve data. **Example:** A user in Delhi queries for customer records, but the data is stored in Mumbai. The system fetches the data without requiring the user to specify the location.
2. **Naming Transparency:** Users can refer to data items using the same name system-wide, regardless of where the data is stored. **Example:** A table named `Customers` is accessed by the same name across all sites.
3. **Replication Transparency:** Users do not need to know whether multiple copies (replicas) of data exist. The DDBMS ensures that all replicas remain consistent and updated. **Example:** If

an inventory database is replicated at multiple warehouses, any update to stock levels at one warehouse is automatically reflected in all replicas.

4. **Fragmentation Transparency:** Users do not need to know if the database is divided into smaller parts (fragments) and stored at different locations. The system automatically reconstructs the data when needed. **Example:** A customer database may be fragmented based on region, with records for customers in North India stored in Delhi and those in South India stored in Bangalore, but users can access the full database without concern for fragmentation.
5. **Local Mapping Transparency:** Hides how data is stored at each local site; users work with a unified view. **Example:** A user accesses the Orders table without knowing its different storage formats at different sites.
6. **Transaction Transparency:** Ensures that a **distributed transaction** (a transaction involving multiple sites) maintains the **ACID** (Atomicity, Consistency, Isolation, Durability) properties. Users perceive transactions as if they are occurring in a single system. **Example:** A bank transfer between two accounts in different branches should appear as a single transaction even if it involves multiple databases.
7. **Performance Transparency:** Users receive optimized query responses regardless of the underlying distribution of data. The system ensures load balancing and efficient data retrieval. **Example:** A hotel booking system retrieves room availability quickly, even if data is stored across multiple cities.

Levels of Transparency: The 3 main levels of transparencies are:

- **Location Transparency**
- **Fragmentation Transparency**
- **Local Mapping Transparency**

Example

Consider an Emp table with the following schema: **Emp (ENAME, EDOB, EAddr, EDept, ESal)**

This employee data is horizontally fragmented based on location and distributed across three sites:

- Fragment E1: Employees in **Delhi**
- Fragment E2: Employees in **Mumbai**
- Fragment E3: Employees in **Kolkata**

- ⇒ Each fragment contains unique records, meaning no employee appears in more than one fragment.
- ⇒ Also, there is no replication — data is only stored at one site.
- ⇒ A user wants to list all employees whose date of birth is earlier than January 1, 1960.

Depending on the level of distribution transparency support we may examine three query cases

Case 1: the database supports fragmentation transparency

Case 2: the database supports location transparency

Case 3: the database supports local mapping transparency

Answer:

Case 1: Fragmentation Transparency

The user is unaware of the fragmentation. The system automatically accesses all fragments.

```
SELECT *  
FROM Emp  
WHERE EDOB < '1960-01-01';
```

✓ Only one query is needed. The DBMS handles fetching from all fragments (E1, E2, E3) automatically.

Case 2: Location Transparency

The user knows the table is distributed across sites but doesn't know the specific site locations.

```
SELECT * FROM E1  
WHERE EDOB < '1960-01-01'  
UNION  
SELECT * FROM E2  
WHERE EDOB < '1960-01-01'  
UNION  
SELECT * FROM E3  
WHERE EDOB < '1960-01-01';
```

✓ Here, the user queries all known locations, and the system handles the connection to each site.

Case 3: Local Mapping Transparency

The user must know exactly where the data is stored and how it's partitioned.

```
SELECT * FROM E1  
WHERE EDOB < '1960-01-01'  
UNION  
SELECT * FROM E2  
WHERE EDOB < '1960-01-01'  
UNION  
SELECT * FROM E3  
WHERE EDOB < '1960-01-01';
```

✓ Here, the user explicitly accesses all fragments (E1, E2, E3) with full awareness of the fragmentation and physical schema.

Data Transparency

Advantages:

- Users don't need to know data location
- Simplifies application development
- Supports data independence
- Offers a centralized view of data

Disadvantages:

- Complex system architecture
- Performance overhead
- High implementation cost

Data Replication

Advantages:

- Improves data availability
- Reduce network traffic
- Faster data access
- Enhances fault tolerance
- Enables load distribution

Disadvantages:

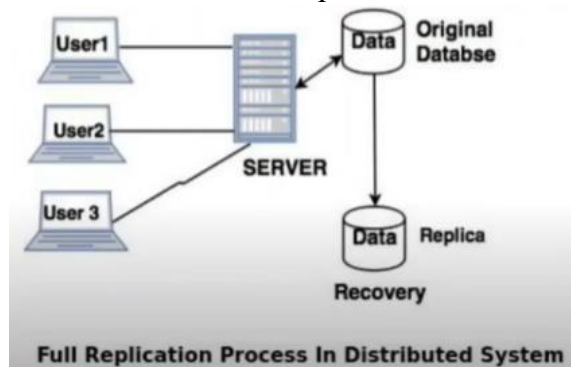
- Complex update management
- More storage needed
- Risk of data inconsistency
- Higher maintenance requirements

Data Replication in Distributed Databases

Data Replication is the process of storing multiple copies of the same data at different locations to improve reliability, availability, and performance. It helps ensure **fault tolerance**, meaning that if one site fails, another site can serve the same data. **Types of Data Replication**

1. **Full Replication:** Every site contains a complete copy of the entire database. Provides high availability and fault tolerance but requires significant storage and synchronization efforts.

Example: A global banking system may fully replicate customer data across all branches so transactions can proceed even if one branch server fails.



Full Replication

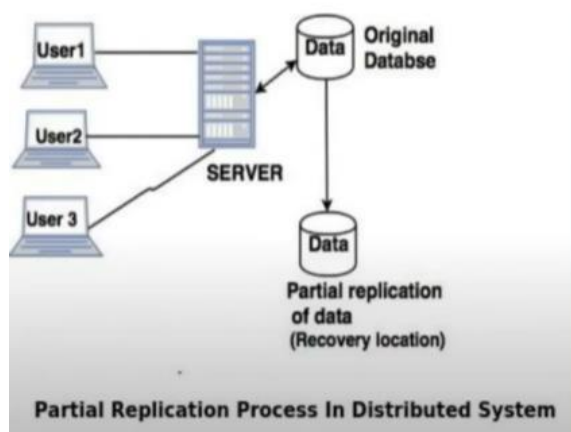
Advantages of full replication

- High availability of data, as database is available to almost every location.
- Faster execution of queries.

Disadvantages of full replication

- Concurrency control is difficult to achieve in full replication.
- Update operation is slower.

2. **Partial Replication:** Only selected parts of the database are replicated at different sites. Reduces storage overhead and improves efficiency while maintaining redundancy. **Example:** An e-commerce platform may store customer order history at a central location but replicate only current orders to regional warehouses.



Partial Replication

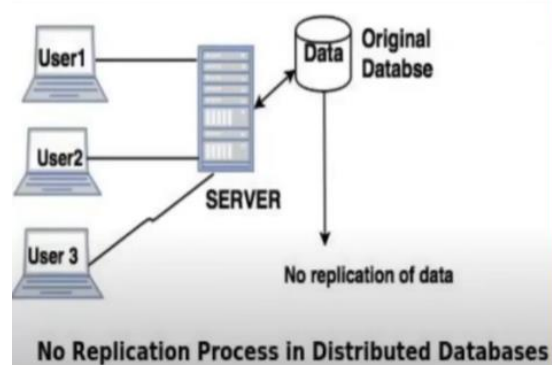
Advantages of Partial replication

- To provide a consistent copy of data across all the database nodes.
- To increase the availability of data.
- The reliability of data is increased through data replication.
- Data Replication supports multiple users and gives high performance

Disadvantages of Partial replication

- More storage space is needed as storing the replicas of same data at different sites consumes more space.
- Data Replication becomes expensive when the replicas at all different sites need to be updated.
- Maintaining Data consistency at all different sites involves complex measures.

3. **No Replication** Data is not duplicated, and each piece of data exists at only one site. Reduces storage requirements but increases the risk of unavailability if a site goes down. **Example:** A university database storing student records in a single centralized location instead of replicating across different campuses.



No Replication

Advantages of no replication

- Concurrency can be minimized.
- Easy recovery of data.

Disadvantages of no replication

- Poor availability of data.
- Slows down the query execution process, as multiple clients are accessing the same server.